Physics and AI Models for Simulations

Quasi

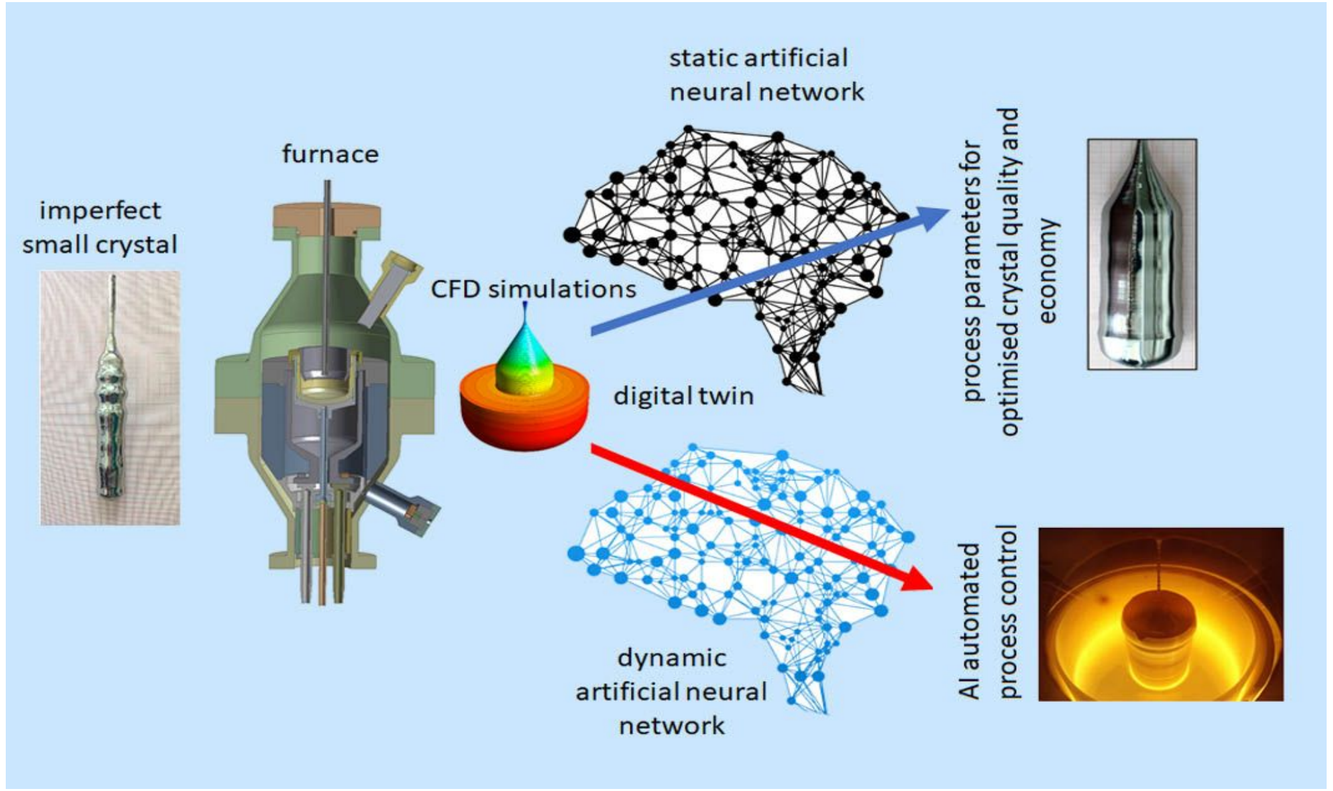
Outline

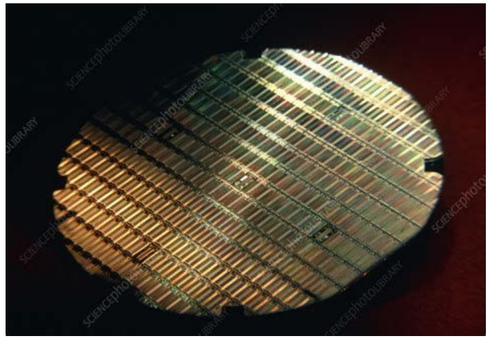
- **Introduction: Domains**
  - **Silicon Crystal Growth Simulations**
  - **Cryogenic Vessels Design**
- **Approaches**
  - **Classical AI - PINN/PINO/GAOT**
- **Outlook**

Silicon Crystal Growth

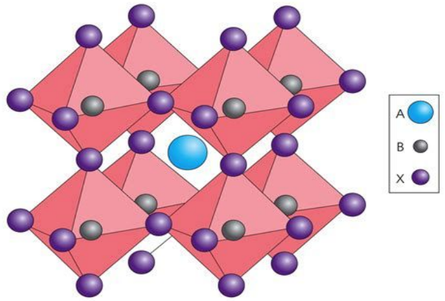
Accelerating CFD Simulations

Real-Time Digital Twins





Silicon Wafers

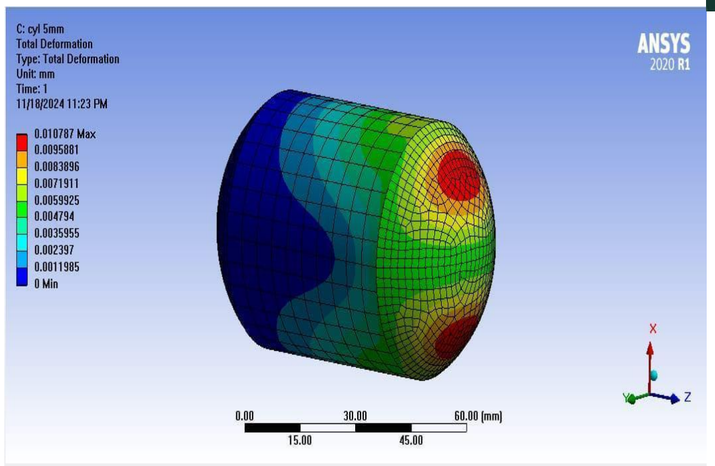


Perovskites

Designing Cryogenic Vessels

Accelerating CFD Reduces Design to Production Time





Green Hydrogen Economy



Collaboration with Indo-Australian Startup

Classical Physics-Inspired AI models

PINNs: Combining Physics

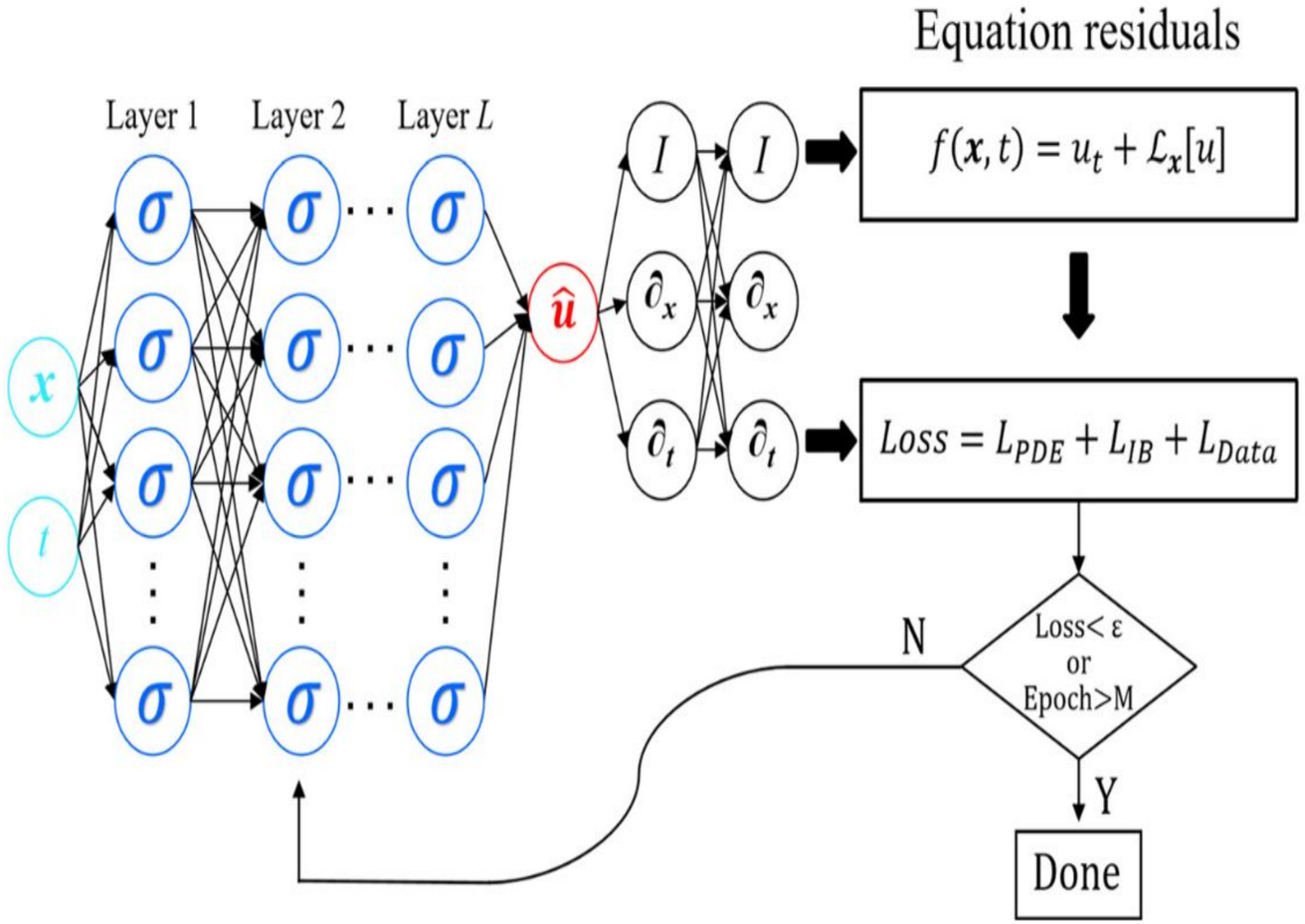


FIG. 1. Schematic diagram of PINN.

Multi-Physics Capability with Less Data or Conditions

Source: https://doi.org/10.1063/5.0226232

PINNs: Combining Physics

PINNs

- Solve PDE of the form

  $\mathcal{L}(\mathbf{u}) = \mathbf{0}$ on domain Ω with BCs and ICs.
- Define trial solution $u_\theta(x)$.
- Minimize loss:

$$\mathcal{L} = \lambda_f \|\mathcal{L}(u_\theta)\|_{\Omega_f}^2 + \lambda_b \|u_\theta - g\|_{\partial\Omega}^2 + \lambda_i \|u_\theta - u_0\|_{t=0}^2$$

Essentially: They optimize a differentiable surrogate that satisfies the PDE.

Multi-Physics Capability with Less Data or Conditions

Source: https://doi.org/10.1063/5.0226232

PINO: Physics Inspired Neural Operators

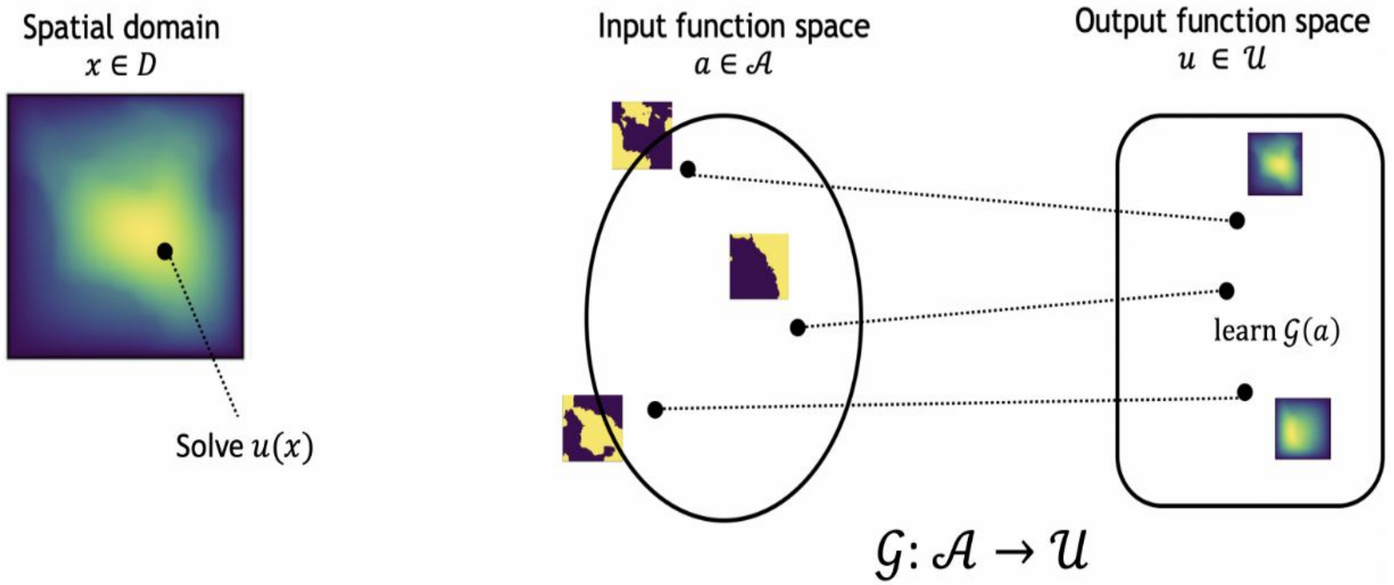


Figure 3: solve for one specific instance verse learn the entire solution operator
Left: numerical solvers and PINNs focus on solving one specific instance. Right: neural operators learn the solution operator for a family of equations.

Higher Accuracy & Multi-Physics Capability with Less Data

PINO: Physics Inspired Neural Operators

PINOs

- Solve families of PDEs:

  $\mathcal{L}$**(u; a) = 0**, parameterized by a (e.g., coefficients, BCs, ICs).
- Learn a neural operator

$$u = \mathcal{G}_\theta(a)$$

- Use integral kernels or spectral convolution layers (FNO, GNO).
- Physics-informed loss includes:

$$\mathcal{L} = \|\mathcal{L}(\mathcal{G}_\theta(a))\|^2 + \text{data terms (optional)}.$$

Key: The operator learns solution *distributions*, not single functions.

Higher Generalisability Data

Comparison

⭐ Why Use PINO Instead of PINNs?

PINO combines the advantages of both:

Feature	PINO	PINN
Learns mappings between *functions*	✔	❌
Good for fast prediction on new inputs	✔	❌
Uses physics constraints	✔	✔
Data-efficiency	High	Medium
Computational cost during inference	Very low	High

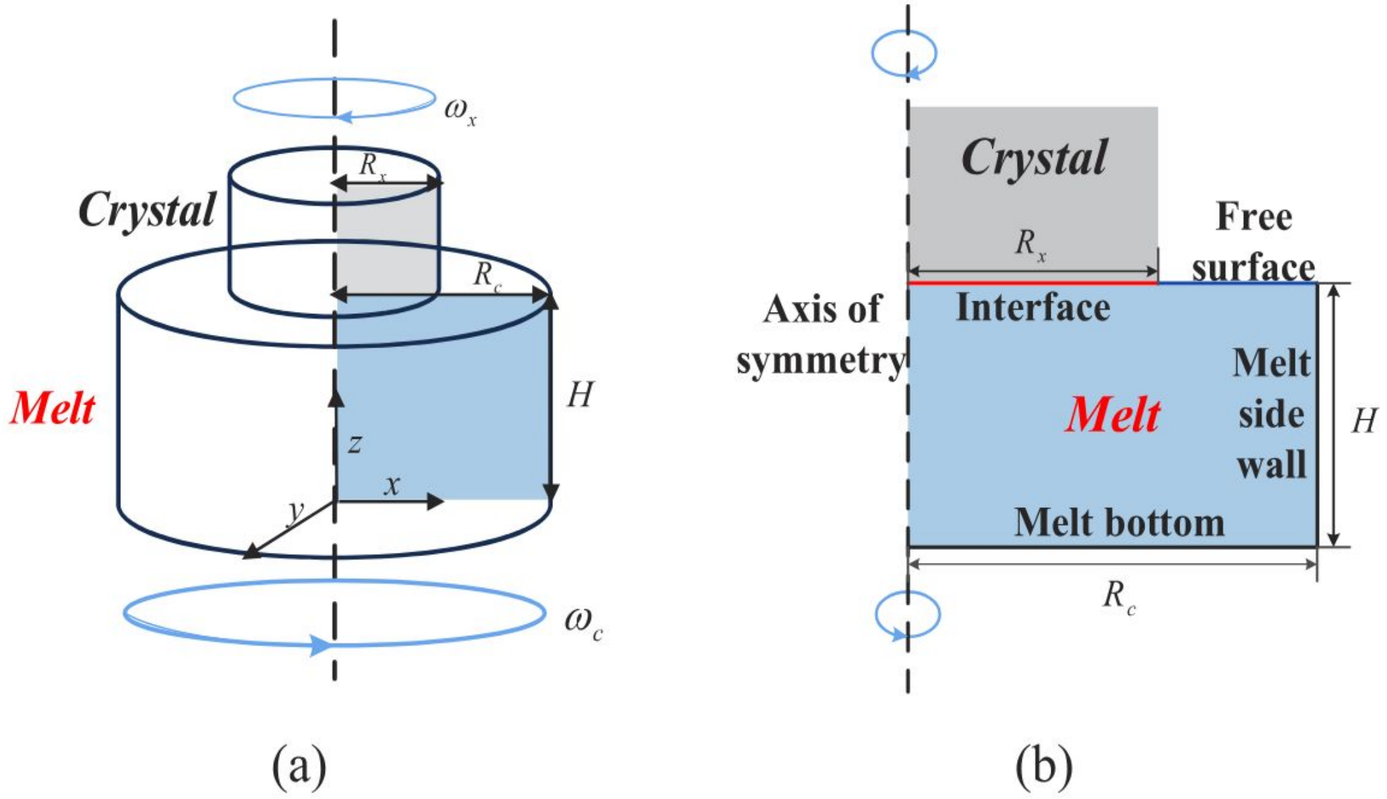
Comparison

5. Generalization Ability

Category	PINNs	PINOs
Generalization to new BCs/ICs	Very limited unless retrained	Very good—operator learns the mapping
Generalization to new geometries	Poor	Better (with geometry-aware operators)
Resolution independence	No	Often yes (esp. Fourier neural operators)

PINOs are explicitly designed for **out-of-distribution generalization** across PDE input conditions.

Silicon Crystal Growth (CZ process)



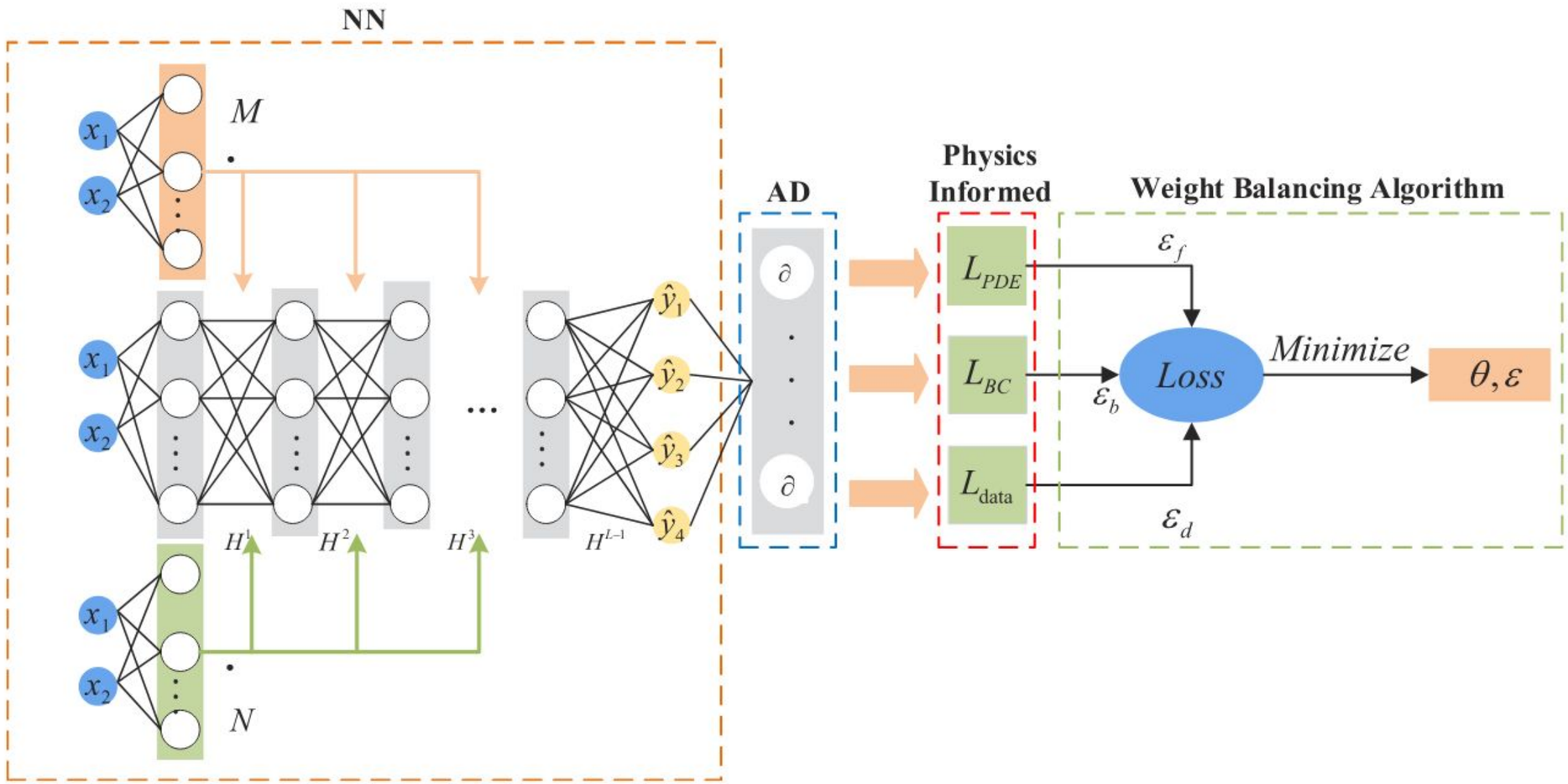
$$\frac{\partial v_r}{\partial r} + \frac{v_r}{r} + \frac{\partial v_z}{\partial z} = 0, \tag{38}$$

$$v_r \cdot \frac{\partial v_r}{\partial r} - \frac{v_\theta^2}{r} + v_z \cdot \frac{\partial v_r}{\partial z} = -\frac{\partial P}{\partial r} + \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 v_r}{\partial r^2} + \frac{\partial v_r}{r\partial r} - \frac{v_r}{r^2} + \frac{\partial^2 v_r}{\partial z^2}\right), \tag{39}$$

Navier–Stokes(NS), Particle Transport, and Heat Transfer equation

Source: doi: 10.1063/5.0271778

Spatial Information & Adaptive Loss Balance PINN



Optimizes the thermal-fluid coupling and Improves training stability

Source: doi: 10.1063/5.0271778

Spatial Information & Adaptive Loss Balance PINN

1. The introduction of spatial coordinate information

Building upon the original multi-layer perceptron (MLP) architecture, the PINN reintroduces the original spatial coordinate information into each hidden layer and incorporates an additional bias vector at each layer to enhance model expressiveness. The updated formulation for the hidden layer is given as

$$H^l = \sigma(W^l H^{l-1} + b^l) \bullet X + b_a, \tag{14}$$

where $\bullet$ represents multiplication by components.

To further enhance the representation of spatial and temporal information, two nonlinear encoders M and N are introduced to replace encoders X and b_a in Eq. (14). Then Eq. (14) is updated as

$$H^l = \sigma(W^l H^{l-1} + b^l) \bullet M + N, \tag{15}$$

where the two encoders are denoted by $M = \sigma(W_M X + b_M)$ and $N = \sigma(W_N X + b_N)$, and the weights and biases of the encoders are represented by W_M, W_N and b_M, b_N, respectively. The improved network architecture is illustrated in the orange box in Fig. 2.

The mathematical formulation of the hidden layer in the SI-LB-PINN architecture is defined as follows:

$$\begin{aligned} H^1 &= \sigma(W^1 X + b^1), \\ Z^l &= \sigma(W^l H^{l-1} + b^l), \\ H^l &= (1 - Z^l) \bullet M + Z^l \bullet N. \\ H^L &= \sigma(W^L H^{L-1} + b^L). \end{aligned} \tag{16}$$

Optimizes the thermal-fluid coupling and Improves training stability

Source: doi: 10.1063/5.0271778

Spatial Information & Adaptive Loss Balance PINN

2. The introduction of weight-balancing algorithms

It is assumed that the output of the neural network follows a Gaussian distribution with mean $\hat{u}$ and variance ε_d, where ε_d is also referred to as the uncertainty parameter. The variance is commonly used to quantify the uncertainty of the output. This leads to

$$p(u|\hat{u}(x,t;\theta)) = N\left(\hat{u}(x,t;\theta), \varepsilon_d^2\right). \tag{17}$$

During training, ε_d is typically used as the weight for the corresponding loss term in the loss function. This weight can be optimized through the maximum likelihood function. Based on the minimization goal of the training process, the negative log-likelihood coefficient of the model is minimized. The expression is formulated as follows:

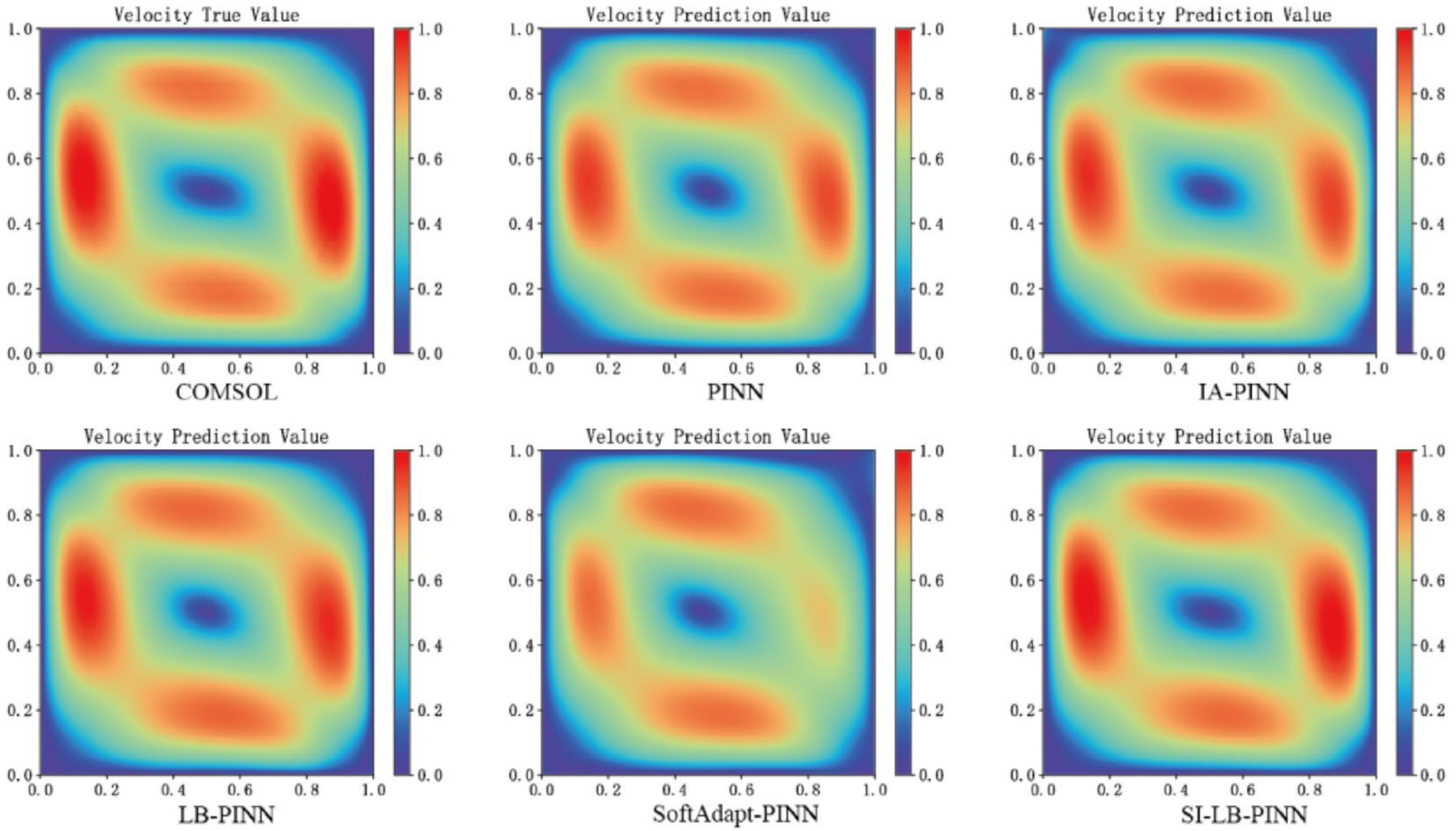
$$\begin{aligned} -\log p(u|\hat{u}(x,t;\theta)) &\propto \frac{1}{2\varepsilon_d^2}\|u-\hat{u}(x,t;\theta)\|^2 + \log \varepsilon_d \\ &= \frac{1}{2\varepsilon_d^2} L_{data}(\theta) + \log \varepsilon_d. \end{aligned} \tag{18}$$

Equation (18) is primarily composed of two components: the residual regression and uncertainty regularization term. In addition, the neural network automatically adjusts the weights of individual loss terms during training by updating the uncertainty parameter ε_d at each iteration, based on maximum likelihood estimation.

Optimizes the thermal-fluid coupling and Improves training stability

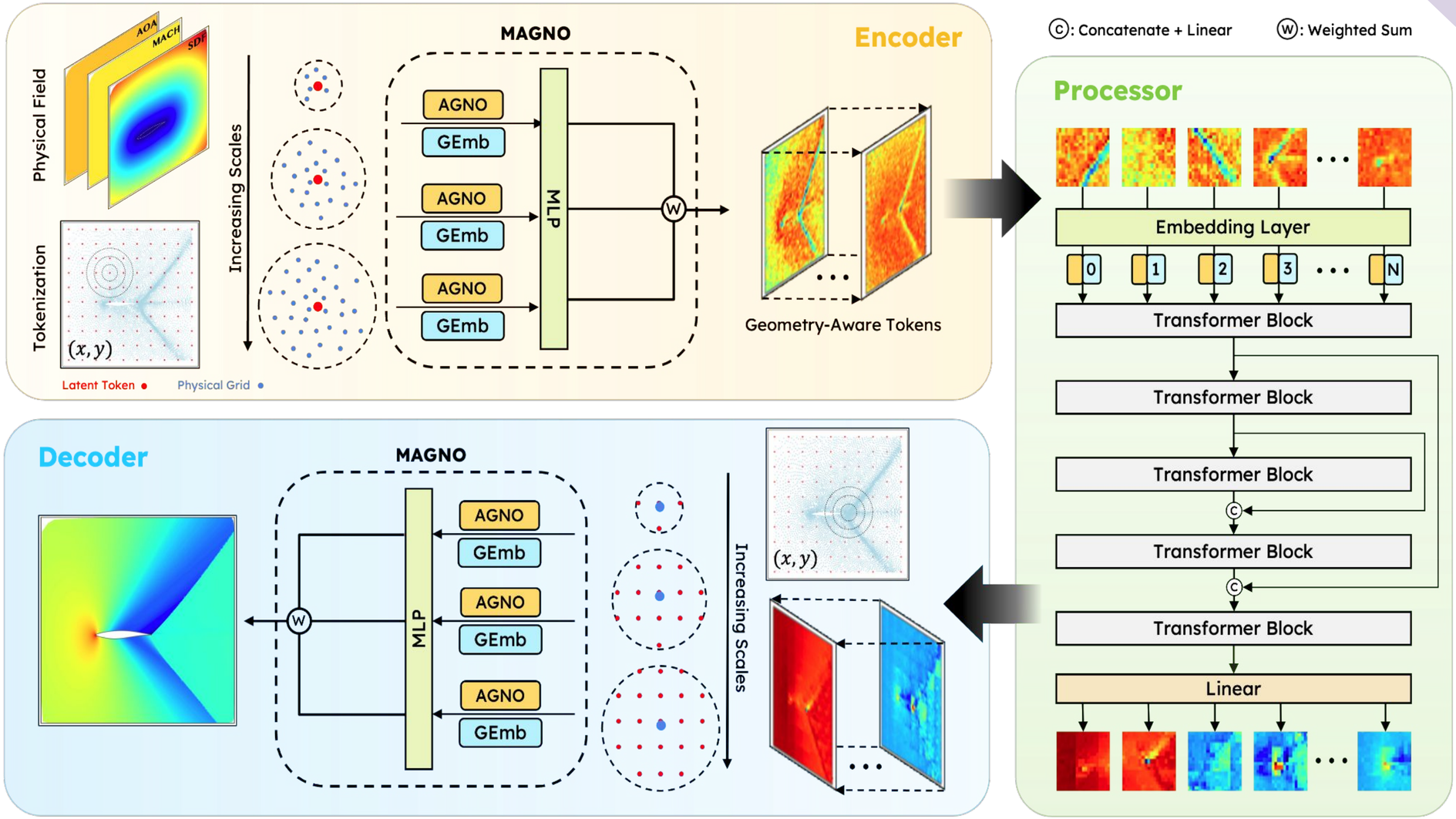
Source: doi: 10.1063/5.0271778

Spatial Information & Adaptive Loss Balance PINN



Source: doi: 10.1063/5.0271778

GAOT: Geometry Aware Operator Transformers



Multiscale Attentional Graph Neural Operator (MAGNO)

Source: arXiv:2505.18781v2 https://camlab-ethz.github.io/GAOT/

Comparison

Aspect	PI-GANO	GAOT (e.g., GINO)
Physics-Informed	Yes (can train without labeled data)	No (typically requires labeled simulation data)
Geometry Handling	Explicit geometry encoder, geometry-aware	SDF and point-cloud encoding, geometry-aware
Data Efficiency	High (physics-informed, less data needed)	Lower (relies on simulation data for training)
Generalization	Across geometry and PDE parameters	Across geometry (parameters less emphasized)
Computational Speed	Fast (neural operator-based)	Extremely fast (demonstrated 26,000x speed-up)

References

Quantum and Physics-Informed Neural Networks

- Afrah Farea, Saiful Khan, & Mustafa Serdar Celebi. (2025). *QCPINN: Quantum-Classical Physics-Informed Neural Networks for Solving PDEs.* arXiv:2503.16678 [quant-ph]. https://arxiv.org/abs/2503.16678
- *Hybrid Quantum PINNs for Computational Fluid Dynamics.* (2025). IOP Science. https://doi.org/10.1088/2632-2153/ad43b2
- *Quantum Physics-Informed Machine Learning for Multiscale Simulations.* (2024). arXiv:2403.08954 [quant-ph]. https://arxiv.org/abs/2403.08954
- *PINNSFORMER: A Transformer-Based Framework for Physics-Informed Neural Networks.* (2023). arXiv:2307.11833v3 [cs.LG].
- Pengpeng Xiao, Muqing Zheng, Anran Jiao, Xiu Yang, & Lu Lu. (2024). *Quantum DeepONet: Neural Operators Accelerated by Quantum Computing.* [Manuscript].
- Siddhant Dutta, Nouhaila Innan, Sadok Ben Yahia, & Muhammad Shafique. (2024). *AQ-PINNs: Attention-Enhanced Quantum Physics-Informed Neural Networks for Carbon-Efficient Climate Modeling.* arXiv:2409.01626v1 [cs.LG].

Physics-Informed Neural Networks in Crystal Growth Simulations

- D. Kolberg, C. Anders, & S. Reke. (2023). *Fast Prediction of Transport Structures in the Melt by Physics-Informed Neural Networks during 'VMCz' Crystal Growth of Silicon. Journal of the Institute of Metals*, https://doi.org/10.1080/00219592.2023.2236656
- J. Ling, Y. Zhang, & D. Chen. (2023). *Simulation of Thermal-Fluid Coupling in Silicon Single Crystal Growth Based on Gradient Normalized Physics-Informed Neural Network. Physics of Fluids.* https://doi.org/10.1063/5.0123811
  (Code repository: https://github.com/callmedrcom/SIPINN)
- Z. Wang, F. Chen, & M. Li. (2024). *RF-PINNs: Reactive Flow Physics-Informed Neural Networks for Field Reconstruction of Laminar and Turbulent Flames. Journal of Computational Physics*, 113698. https://doi.org/10.1016/j.jcp.2024.113698
- Y. Zhao, H. Liu, & J. Xu. (2025). *Research on the Thermal-Fluid Coupling in the Growth Process of Czochralski Silicon Single Crystals Based on an Improved Physics-Informed Neural Network. AIP Advances, 15*(10), 105202. https://doi.org/10.1063/5.0271778

Thank You

Name: A Seshaditya
Email: aditya@quasi.digital
Websites: https://quasi.digital
https://github.com/adytiaa/quasi.ai
https://adytiaa.github.io/quasi.ai/